

Lab 3: The Average Treatment Effect

Federico Bertoia

Ghent University

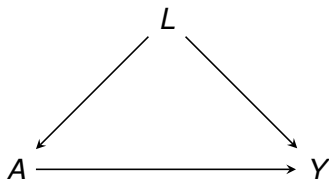
Department of Applied Mathematics, Computer Science and Statistics

With thanks to Kelly Van Lancker, Pawel Morzywolek, Oliver Dukes, Karel Vermeulen and Sjouke Vandenberghe

Background

- Observational study of 996 patients.
- **Exposure**: abciximab. 1=augmented treatment (blood thinner), 0=standard PCI treatment (open arteries with balloon).
- **Outcome**: Survival longer than 6 months.
- Exposure not randomised → more severely ill patients assigned to treatment with abciximab.

Setting



Where

- A : exposure, which we will consider to be binary,
- L : (possibly high-dimensional) set of confounders,
- Y : outcome.

Preliminaries

R Code

```
lindner$survival_binary<-ifelse(lindner$sixMonthSurvive==FALSE,1,0)  
table(lindner$survival_binary, lindner$abcix)
```

Survival	Abciximab	
	No	Yes
Yes	283	687
No	15	11

26 deaths, 11 among the augmented treatment group.

Question 1 - Risk Difference

Risk difference: using linear regression

$$E(Y|A) = \alpha + \beta A$$

R Code

```
unadjusted = survival_binary ~ abcix  
model <- lm(unadjusted, data = lindner)  
summary(model)  
confint(model)
```

Question 1 - Risk Difference

Output

```
> summary(model)
```

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	0.05034	0.00920	5.471	5.66e-08	***
abcix	-0.03458	0.01099	-3.146	0.0017	**

```
> confint(model)
```

	2.5 %	97.5 %
(Intercept)	0.03228184	0.06838930
abcix	-0.05614223	-0.01301029

Question 1 - Risk Difference

Interpretation?

Question 1 - Risk Difference

Interpretation?

- We find an **average reduction of 3.46 %**
($p = 0.0017$, $SE = 0.011$) in risk of death within 6 months.

Question 1 - Risk Difference

Interpretation?

- We find an **average reduction of 3.46 %**
($p = 0.0017$, $SE = 0.011$) in risk of death within 6 months.
- Receiving augmented treatment is thus **associated** with an average reduction in risk of 3.46% as compared to receiving the standard treatment and a 95% confidence interval of $[-0.056, -0.013]$

Question 1 - Risk Difference

Interpretation?

- We find an **average reduction of 3.46 %** ($p = 0.0017$, $SE = 0.011$) in risk of death within 6 months.
- Receiving augmented treatment is thus **associated** with an average reduction in risk of 3.46% as compared to receiving the standard treatment and a 95% confidence interval of $[-0.056, -0.013]$

Causal?

Question 1 - Risk Difference

Interpretation?

- We find an **average reduction of 3.46 %** ($p = 0.0017$, $SE = 0.011$) in risk of death within 6 months.
- Receiving augmented treatment is thus **associated** with an average reduction in risk of 3.46% as compared to receiving the standard treatment and a 95% confidence interval of $[-0.056, -0.013]$

Causal?

No due to **confounding**!

Question 1 - Odds Ratio

Odds ratio: using logistic regression

$$\begin{aligned}P(Y = 1|A) &= \text{expit}(\alpha^* + \beta^* A) \\&= \frac{\exp(\alpha^* + \beta^* A)}{1 + \exp(\alpha^* + \beta^* A)}\end{aligned}$$

or alternatively,

$$\text{logit}P(Y = 1|A) = \alpha^* + \beta^* A$$

$$\log\left(\frac{P(Y = 1|A)}{P(Y = 0|A)}\right) = \alpha^* + \beta^* A$$

R Code

```
logit_model <- glm(unadjusted, data = lindner, family = 'binomial')
```

Question 1 - Odds Ratio

$$\begin{aligned}\widehat{OR} &= \frac{\text{odds}(Y = 1 \mid A = 1)}{\text{odds}(Y = 1 \mid A = 0)} = \frac{P(Y = 1 \mid A = 1)}{P(Y = 0 \mid A = 1)} \times \frac{P(Y = 0 \mid A = 0)}{P(Y = 1 \mid A = 0)} \\ &= \frac{\exp(\hat{\alpha}^* + \hat{\beta}^*)}{\exp(\hat{\alpha}^*)} \\ &= \exp(\hat{\beta}^*) = 0.302 \quad 95\%CI : \quad [0.13; 0.66]\end{aligned}$$

R Code

```
odds_ratio = exp(logit_model$coefficients[2])  
odds_ratio_LB = exp(confint(logit_model)[2,1])  
odds_ratio_UB = exp(confint(logit_model)[2,2])
```

Question 1 - Odds Ratio

Odds ratio: using logistic regression

- We find an odds ratio of 0.302 .
- This means that the odds of dying in the augmented treatment group is 0.3 times the odds of dying in the standard treatment group.

Question 2 - G-Computation

Outcome is assumed to be generated according to some unknown model, for example:

$$\begin{aligned}E(Y \mid A, \mathbf{L}) &= Q(A, \mathbf{L}) \\P(Y = 1 \mid A, \mathbf{L}) &= Q(A, \mathbf{L})\end{aligned}$$

where we use $Q(\bullet)$ to emphasize that the expected outcome conditional on A and $\mathbf{L}$ need not to be considered standard linear or logistic regression functions.

$$\hat{\psi}_{gComp} = \frac{1}{N} \sum_{i=1}^N \left\{ \hat{Q}_i(A = 1, \mathbf{L}) - \hat{Q}_i(A = 0, \mathbf{L}) \right\}$$

Question 2 - G-Computation (a)

- 1 Fit a model for Y as a function of A and L

$$P(Y = 1|A, L) = \text{expit}(\beta_0 + \beta_1 A + \beta_2 L)$$

- 2 Obtain prediction for *all* patients for $A=1$ and their observed L .

$$\hat{P}(Y^1 = 1|L) = \text{expit}(\hat{\beta}_0 + \hat{\beta}_1 + \hat{\beta}_2 L)$$

- 3 Take the mean of these predictions

$$\hat{P}(Y^1 = 1) = \frac{1}{n} \sum_{i=1}^n \text{expit}(\hat{\beta}_0 + \hat{\beta}_1 + \hat{\beta}_2 L_i)$$

- 4 Repeat for $A = 0$.

Question 2 - G-Computation (a)

R code

```
Y <- lindner$survival_binary
A <- lindner$abcix
X <- lindner[, c("stent", "height", "female", "diabetic", "acutemi",
                "ejecfrac", "veslproc")]

dat <- as.data.frame(cbind(Y, A, X))
formula_gcomp <- as.formula(paste("Y ~ A +", paste(colnames(X),
                                                    collapse = " + ")))

# G-Computation By Hand using parametric model (a)
fit <- glm(formula_gcomp, family = 'binomial', data = dat)

dat0 <- dat; dat0$A <- 0
dat1 <- dat; dat1$A <- 1

Q0 <- predict(fit, newdata = dat0, type = "response")
Q1 <- predict(fit, newdata = dat1, type = "response")

ate_gcomp_byhand <- mean(Q1 - Q0); ate_gcomp_byhand
```

Question 2 - G-Computation (a)

$$\hat{E}(Y^1 - Y^0) = \hat{P}(Y^1 = 1) - \hat{P}(Y^0 = 1) = -0.0459$$

The probability of dying within 6 months is **4.59 %** less for the treated relative to the controls.

Question 2 - G-Computation (b)

How to get the standard error of $\hat{E}(Y^1 - Y^0)$?

- In 'stdReg'/'stdGLM', a **sandwich** estimator is used to estimate the variance covariate matrix for $\{\hat{E}(Y^1), \hat{E}(Y^0)\}$.

$$\begin{aligned}\text{var}\{\hat{E}(Y^1 - Y^0)\} &= \text{var}\{\hat{E}(Y^1) - \hat{E}(Y^0)\} \\ &= \text{var}\{\hat{E}(Y^1)\} + \text{var}\{\hat{E}(Y^0)\} - 2\text{cov}\{\hat{E}(Y^1), \hat{E}(Y^0)\}\end{aligned}$$

- Alternatively, can use the **bootstrap**.

Question 2 - G-Computation (b)

R Code

```
# G-Computation using stdReg  
fit <- glm(formula_gcomp, family = 'binomial', data = dat)  
fit.std <- stdGlm(fit = fit, data = dat, X = "A")  
print(summary(fit.std))
```

Output

```
> print(summary(fit.std))
```

Formula: Y ~ A + stent + height + female + diabetic + acutemi +
 ejecfrac + veslproc

Family: binomial

Link function: logit

Exposure: A

	Estimate	Std. Error	lower 0.95	upper 0.95
0	0.0605	0.0163	0.02853	0.0924
1	0.0146	0.0045	0.00576	0.0234

Question 2 - G-Computation (b)

R Code

```
ate_gcomp <- fit.std$est[2] - fit.std$est[1]
ate_gcomp_se <- sqrt(fit.std$vcov[1] + fit.std$vcov[4]
                    - 2* fit.std$vcov[3])
```

```
ate_gcomp
ate_gcomp_se
```

Output

```
> ate_gcomp
[1] -0.04589919
> ate_gcomp_se
[1] 0.01731326
```

Question 2 - G-Computation (b - alternative)

R Code

```
# G-Computation Parametric with functions (b - alternative)

muhat_par <- estimate_mu_par(Y = Y,
                             A = A,
                             X = X,
                             family = 'binomial')

ate_gcomp_par <- est_gcomp(muhat = muhat_par)
ate_gcomp_par_se <- bootstrap_se_gcomp(Y = Y, A = A, X = X,
                                       family = 'binomial', B = 500)

ate_gcomp_par
ate_gcomp_par_se
```

Output

```
> ate_gcomp_par
[1] -0.04589919
> ate_gcomp_par_se
[1] 0.01718875
```

Question 2 - G-Computation (c)

Should we trust our estimates?

Question 2 - G-Computation (c)

Should we trust our estimates?

If $Q(A, \mathbf{L})$ is parametric, yes.

$$Q(A, \mathbf{L}) = \text{expit}(\beta_0 + \beta_1 A + \beta_2' \mathbf{L})$$

Question 2 - G-Computation (c)

Should we trust our estimates?

If $Q(A, \mathbf{L})$ is parametric, yes.

$$Q(A, \mathbf{L}) = \text{expit}(\beta_0 + \beta_1 A + \beta_2' \mathbf{L})$$

If not, then our model is **misspecified**.

Question 2 - G-Computation (d)

R Code

```
# G-Computation Non Parametric with SuperLearner (d)
library(SuperLearner)
sl_lib <- list("SL.glm", "SL.step", "SL.glm.interaction", "SL.randomForest")

AX <- as.data.frame(cbind(A, X))

Q.sl <- SuperLearner(Y = Y,
                    X = AX,
                    SL.library = sl_lib,
                    family = 'binomial')

Q1 <- predict(Q.sl, newdata = data.frame(A = 1, X))$pred
Q0 <- predict(Q.sl, newdata = data.frame(A = 0, X))$pred

ate_gcomp_sl <- mean(Q1 - Q0)
```

Question 2 - G-Computation (d)

Output

```
> ate_gcomp_sl  
[1] -0.04019047
```

```
> Q.sl$coef  
SL.glm_All    SL.step_All    SL.glm.interaction_All    SL.randomForest_All  
0.7733495      0.0000000      0.0000000      0.2266505
```

Question 2 - G-Computation (d - alternative)

R Code

```
# G-Computation Non Parametric with functions and
# cross-fitting (d - alternative)

muhat_est_nonpar <- estimate_mu_nonpar(Y = Y,
                                       A = A,
                                       X = X,
                                       sl_lib = sl_lib,
                                       nsplits = 5,
                                       family = 'binomial')

muhat_nonpar <- muhat_est_nonpar$preds
coefs_gcomp <- muhat_est_nonpar$coefs

ate_gcomp_nonpar <- est_gcomp(muhat = muhat_nonpar)

ate_gcomp_nonpar_se <- bootstrap_se_gcomp(Y = Y, A = A, X = X,
                                           family = 'binomial', B = 50, nonpar = TRUE, sl_lib = sl_lib,
                                           nsplits = 3, stratified = FALSE)
```

Question 2 - G-Computation (d)

Output

```
> coefs_gcomp

SL.glm_All      SL.step_All      SL.glm.interaction_All  SL.randomForest_All
0.185724832     0.555596872           0.007123826           0.251554471
>
> ate_gcomp_nonpar
[1] -0.03995951

> ate_gcomp_nonpar_se
[1] 0.01432095
```

Should we trust our estimates?

Question 2 - G-Computation (e)

Pros

- We do not have the problem of misspecification.
- Able to capture complex relationships in the data.

Cons

- Nonparametric machine learning methods suffer from **plug-in bias**, which do not vanish as sample size increase...
- Slow convergence rates.
- Lack of valid confidence intervals.

Question 2 - G-Computation (f)

It is not possible to get a valid standard error and confidence intervals, not even with the bootstrap!

Question 3 - IPW

Outcome is assumed to be generated according to some unknown model, for example:

$$P(A = 1 \mid \mathbf{L}) = g(\mathbf{L})$$

$g(\mathbf{L})$ is known as the propensity score.

$$\hat{\psi}_{ipw} = \frac{1}{N} \sum_{i=1}^N \left\{ \frac{A_i Y_i}{\hat{g}_i(L)} - \frac{(1 - A_i) Y_i}{1 - \hat{g}_i(L)} \right\}.$$

Question 3 - IPW (a)

R Code

```
# IPW using a parametric model and then a weighted linear regression (a)
library(sandwich)

AX <- as.data.frame(cbind(A, X))

formula_g <- as.formula(paste("A ~", paste(colnames(X), collapse = " + "))
ghat <- glm(formula_g, family = 'binomial', AX)$fitted.values

ghat_bounded = bound_pi(ghat)

weights <- (A/ghat_bounded) + ((1-A)/(1-ghat_bounded))

fit_ipw <- lm(Y ~ A, weights = weights)

ate_ipw <- fit_ipw$coefficients[2]
ate_ipw_se <- sqrt(vcovHC(fit_ipw, type = "HC")[2,2])
```

Question 3 - IPW (a)

Output

```
> ate_ipw  
-0.06609299  
> ate_ipw_se  
[1] 0.02962059
```

Same interpretation as before: we find a decrease in the probability of dying within 6 months by **6.61 %** in the treated relative to the controls.

Should we trust these results?

Question 3 - IPW (a - alternative)

R Code

```
# IPW parametric with functions (a - alternative)

pihat_par <- estimate_pi_par(A=A, X=X)
pihat_par_bounded <- bound_pi(pihat_par)

fit_ipw_par <- est_ipw(Y = Y,
                      A = A,
                      pi = pihat_par_bounded)

ate_ipw_par <- fit_ipw_par[1]
ate_ipw_par_sd <- fit_ipw_par[2]
```

Output

```
ate_ipw_par
      est
-0.06609299
ate_ipw_par_sd
      se
0.02962059
```

Question 3 - IPW (b)

R Code

```
# IPW using a SuperLearner for propensity scores and then  
# a weighted linear regression (b)  
  
g.sl <- SuperLearner(Y = A,  
                    X = X,  
                    SL.library = sl_lib,  
                    family = 'binomial')  
  
ghat_sl <- predict(g.sl, newdata = X)$pred  
  
ghat_sl_bounded <- bound_pi(ghat_sl)  
  
weights_sl <- (A/ghat_sl_bounded) + ((1-A)/(1-ghat_sl_bounded))  
  
fit_ipw_sl <- lm(Y ~ A, weights = weights_sl)  
  
ate_ipw_nonpar <- fit_ipw_sl$coefficients[2]  
ate_ipw_nonpar_se <- sqrt(vcovHC(fit_ipw_sl, type = "HC")[2,2])
```

Question 3 - IPW (b)

Output

```
> ate_ipw_nonpar
      A
-0.04193259
> ate_ipw_nonpar_se
[1] 0.01694491
```

Question 3 - IPW (b - alternative)

R Code

```
# IPW nonparametric (SuperLearner) with functions (b - alternative)

pihat_nonpar_fit <- estimate_pi_nonpar(A = A,
                                       X = X,
                                       sl_lib = sl_lib,
                                       nsplits = 5)

pihat_nonpar <- pihat_nonpar_fit$preds

pihat_nonpar_bounded <- bound_pi(pihat_nonpar)

fit_ipw_nonpar <- est_ipw(Y = Y,
                         A = A,
                         pi = pihat_nonpar_bounded)

ate_ipw_nonpar <- fit_ipw_nonpar[1]
ate_ipw_nonpar_se <- fit_ipw_nonpar[2]
```

Question 3 - IPW (b - alternative)

Output

```
> ate_ipw_nonpar <- fit_ipw_nonpar[1]; ate_ipw_nonpar
      est
-0.05476337
> ate_ipw_nonpar_sd <- fit_ipw_nonpar[2]; ate_ipw_nonpar_sd
      se
0.0238114
```

ATE and SE are different due to sample splitting.
Again, should we rely on them?

Question 3 - IPW (c)

R Code

```
# Check positivity assumption (c)
Apihat = as.data.frame(cbind(A, pihat_nonpar_bounded))
#here we are using estimated propensity scores with cross-fitting

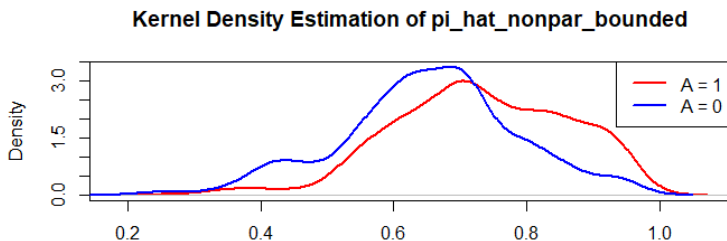
pi_A1 <- Apihat$pihat_nonpar_bounded[Apihat$A == 1]
pi_A0 <- Apihat$pihat_nonpar_bounded[Apihat$A == 0]

pi_A1_inverse <- (1/Apihat$pihat_nonpar_bounded[Apihat$A == 1])
pi_A0_inverse <- (1/Apihat$pihat_nonpar_bounded[Apihat$A == 0])
```

Question 3 - IPW (c)

R Code

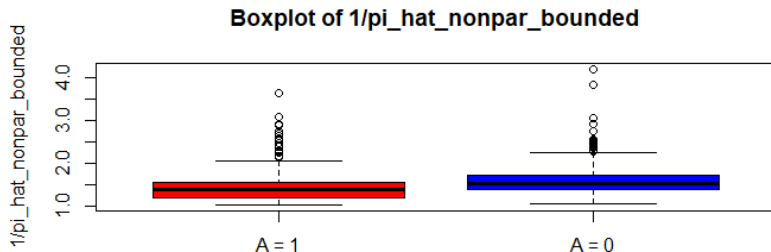
```
plot(density(pi_A1, na.rm = TRUE), col = "red", lwd = 2,  
     main = "Kernel Density Estimation of pi_hat_nonpar_bounded",  
     xlab = "pi_hat_nonpar_bounded",  
     ylim = c(0, max(density(pi_A1)$y, density(pi_A0)$y)))  
  
lines(density(pi_A0, na.rm = TRUE), col = "blue", lwd = 2)  
  
legend("topright", legend = c("A = 1", "A = 0"),  
      col = c("red", "blue"), lwd = 2)
```



Question 3 - IPW (c)

R Code

```
boxplot(pi_A1_inverse, pi_A0_inverse,  
        names = c("A = 1", "A = 0"),  
        col = c("red", "blue"),  
        main = "Boxplot of 1/pi_hat_nonpar_bounded",  
        ylab = "1/pi_hat_nonpar_bounded")
```



Question 4 - AIPW

$$\hat{\Psi}_{\text{aipw}} = \frac{1}{N} \sum_{i=1}^N \left\{ \frac{A_i}{\hat{g}_i(L)} - \frac{1 - A_i}{1 - \hat{g}_i(L)} \right\} \{ Y_i - \hat{Q}(A, \mathbf{L}) \} + \hat{Q}_i(A = 1, \mathbf{L}) - \hat{Q}_i(A = 0, \mathbf{L})$$

- It is a **doubly robust** estimator, meaning it converges to the true value as the sample size grows if either the propensity score ($g(L)$) or the outcome model ($Q(A, L)$) are consistently estimated.
- Moreover, if both are consistently estimated, it is asymptotically efficient, i.e, it has the smallest possible asymptotic variance.

Question 4 - AIPW

Differently from G-Computation and IPW, which are singly robust estimators, AIPW does not suffer from their limitations. In particular:

- AIPW is $\sqrt{N}$ -consistent
- Asymptotically normal
- (Possibly) efficient

even if g and Q converge at slower nonparametric rates.

Question 4 - AIPW (a + b)

Remember from the theory class that the efficient influence curve of the ATE is:

$$\begin{aligned} \mathbb{IF}(\text{ATE}) = & \left\{ \frac{A}{P(A=1 | L)} - \frac{1-A}{P(A=0 | L)} \right\} \{Y - E(Y | A, L)\} \\ & + E(Y | A=1, L) - E(Y | A=0, L) - E(Y^1) - E(Y^0) \end{aligned}$$

and that the variance of the estimator can therefore be estimated as 1 over n times the sample variance of the (estimated) influence curves.

Question 4 - AIPW (a + b)

R Code

```
N <- length(A)

# AIPW (a + b)
g1_aipw <- pihat_nonpar_bounded
g0_aipw <- 1-pihat_nonpar_bounded
Q1_aipw <- muhat_nonpar$mu1
Q0_aipw <- muhat_nonpar$mu0
Q_aipw <- muhat_nonpar$mu

ate_aipw_by_hand <- mean(((A/g1_aipw) - (1-A)/g0_aipw)*(Y - Q_aipw) +
                          Q1_aipw - Q0_aipw)

influence_curve_aipw <- ((A/g1_aipw) - (1-A)/g0_aipw)*(Y - Q_aipw) +
                        Q1_aipw - Q0_aipw - ate_aipw_by_hand

ate_aipw_by_hand_se <- (1/sqrt(N)) * sd(influence_curve_aipw)

ate_aipw_by_hand
ate_aipw_by_hand_se
```

Question 4 - AIPW (a + b)

Output

```
> ate_aipw_by_hand  
[1] -0.05519799  
> ate_aipw_by_hand_se  
[1] 0.02548664
```

Question 4 - AIPW (c)

R Code

```
library(devtools) #install.packages("devtools")
library(npcausal) #install_github("ehkennedy/npcausal")

aipw_1cv <- ate(y = Y, a = A, x = X, nsplits = 1, sl.lib = sl_lib)

aipw_2cv <- ate(y = Y, a = A, x = X, nsplits = 2, sl.lib = sl_lib)

aipw_5cv <- ate(y = Y, a = A, x = X, nsplits = 5, sl.lib = sl_lib)
```

Note that the `ate` function from the `npcausal` package assumes a continuous outcome Y (if you look at how it is coded in the documentation, there is the option `family = 'gaussian'`).

Question 4 - AIPW (c)

Output

```
aipw_1cv <- ate(y = Y, a = A, x = X, nsplits = 1, sl.lib = sl_lib)
```

	parameter	est	se	ci.ll	ci.ul	pval
1	E{Y(0)}	0.06512411	0.016403495	0.032973258	0.09727496	0.000
2	E{Y(1)}	0.01531991	0.004789813	0.005931873	0.02470794	0.001
3	E{Y(1)-Y(0)}	-0.04980420	0.017053658	-0.083229372	-0.01637903	0.003

```
>  
> aipw_2cv <- ate(y = Y, a = A, x = X, nsplits = 2, sl.lib = sl_lib)
```

	parameter	est	se	ci.ll	ci.ul	pval
1	E{Y(0)}	0.07788996	0.031595549	0.015962686	0.139817238	0.014
2	E{Y(1)}	0.01654307	0.004901613	0.006935905	0.026150227	0.001
3	E{Y(1)-Y(0)}	-0.06134690	0.031946191	-0.123961429	0.001267637	0.055

```
>  
> aipw_5cv <- ate(y = Y, a = A, x = X, nsplits = 5, sl.lib = sl_lib)
```

	parameter	est	se	ci.ll	ci.ul	pval
1	E{Y(0)}	0.07267774	0.027482113	0.018812794	0.126542677	0.008
2	E{Y(1)}	0.01513901	0.004828043	0.005676041	0.024601972	0.002
3	E{Y(1)-Y(0)}	-0.05753873	0.027866891	-0.112157835	-0.002919623	0.039

Question 4 - AIPW (c)

Note that the `ate` function from the `npcausal` package assumes a continuous outcome Y (if you look at how it is coded in the documentation, there is the option `family = 'gaussian'` for estimating the outcome model which you cannot change).

A better alternative for our specific case could be using the function `est_aipw`. With it, you can correctly deal with a binary outcome just by estimating the argument `muhat_nonpar` using the function `estimate_mu_nonpar` using `family = 'binomial'`.

Question 4 - AIPW (c - alternative)

R Code

```
# AIPW, with functions (c - alternative)
ate_aipw_nonpar_est <- est_aipw(Y, A, pihat_nonpar_bounded, muhat_nonpar)

ate_aipw_nonpar <- ate_aipw_nonpar_est[1]; ate_aipw_nonpar
ate_aipw_nonpar_se <- ate_aipw_nonpar_est[2]; ate_aipw_nonpar_se
```

Output

```
> ate_aipw_nonpar <- ate_aipw_nonpar_est[1]; ate_aipw_nonpar
      est
-0.05519799
> ate_aipw_nonpar_se <- ate_aipw_nonpar_est[2]; ate_aipw_nonpar_se
      se
0.02548664
```

Note that this ATE and SE are exactly the same as point (a+b), since we used the same nuisance parameters estimates are input to compute them.

Question 4 - AIPW (d)

We already discussed the advantages of the AIPW. However, we should also say that:

- The validity of AIPW relies on the positivity assumption

In other words, if we have small overlapping between treated and control in the propensity score, then we get high variable ATE estimates with high standard error.

Question 5 - TMLE

$$\hat{\Psi}_{tmle} = \frac{1}{N} \sum_{i=1}^N \left\{ \hat{Q}_i^{tmle}(A=1, \mathbf{L}) - \hat{Q}_i^{tmle}(A=0, \mathbf{L}) \right\}$$

- It removes the plug-in bias by **updating** the initial predictions.

Question 5 - TMLE (a + b)

R Code: Step 1. Make initial predictions

```
# TMLE (a + b)
Q.sl <- SuperLearner(Y = Y,
                    X = AX,
                    SL.library = sl_lib,
                    family = 'binomial')

g.sl <- SuperLearner(Y = A,
                    X = X,
                    SL.library = sl_lib,
                    family = 'binomial')

Q1 <- predict(Q.sl, newdata = data.frame(A = 1, X))$pred
Q0 <- predict(Q.sl, newdata = data.frame(A = 0, X))$pred

Q <- A*Q1 + (1-A)*Q0
g <- predict(g.sl, newdata = X)$pred
```

Question 5 - TMLE (a + b)

R Code: Step 2. Construct the clever covariates

```
H1 <- A/g
```

```
H0 <- (1-A) / (1-g)
```

```
H <- H1-H0
```

They are chosen to set to zero

$$0 = \frac{1}{n} \sum_{i=1}^n \frac{A_i}{g_n(L_i)} \{Y_i - \bar{Q}_{1n}(L_i)\}$$

$$0 = \frac{1}{n} \sum_{i=1}^n \frac{1 - A_i}{1 - g_n(L_i)} \{Y_i - \bar{Q}_{0n}(L_i)\}$$

Question 5 - TMLE (a + b)

R Code: Step 3. Construct a parametric model around the initial predictions and update them.

```
# Fluctuation Model 1

# Estimate parameters delta in the fluctuation models
delta1_tmle1 <- coef(glm(Y ~ -1 + offset(qlogis(Q1)) + H1,
                        family = 'binomial'))
delta0_tmle1 <- coef(glm(Y ~ -1 + offset(qlogis(Q0)) + H0,
                        family = 'binomial'))

# Targeting step
Q1_tmle1 <- plogis(qlogis(Q1) + delta1_tmle1/g)
Q0_tmle1 <- plogis(qlogis(Q0) + delta0_tmle1/(1-g))
```

Question 5 - TMLE (a + b)

R Code: Step 3. Construct a parametric model around the initial predictions and update them.

```
# Fluctuation Model 1

# Estimate parameters delta in the fluctuation models
delta1_tmle1 <- coef(glm(Y ~ -1 + offset(qlogis(Q1)) + H1,
                        family = 'binomial'))
delta0_tmle1 <- coef(glm(Y ~ -1 + offset(qlogis(Q0)) + H0,
                        family = 'binomial'))

# Targeting step
Q1_tmle1 <- plogis(qlogis(Q1) + delta1_tmle1/g)
Q0_tmle1 <- plogis(qlogis(Q0) + delta0_tmle1/(1-g))
```

Question 5 - TMLE (a + b)

R Code: Step 4. Compute the ATE as a G-computation using updated predictions.

```
# Compute  $E[Y^1]$  and  $E[Y^0]$ 
EY1_tmle1 <- mean(Q1_tmle1); EY1_tmle1
EY0_tmle1 <- mean(Q0_tmle1); EY0_tmle1
ATE_tmle1 <- EY1_tmle1 - EY0_tmle1; ATE_tmle1

# Compute standard errors of  $E[Y^1]$  and  $E[Y^0]$ 
# using the efficient influence curves
EIC_EY1_tmle1 <- (A / g) * (Y - Q1_tmle1) + Q1_tmle1 - EY1_tmle1
EIC_EY0_tmle1 <- ((1-A) / (1-g)) * (Y - Q0_tmle1) + Q0_tmle1 - EY0_tmle1

se_EY1_tmle1 <- (1/sqrt(N)) * sd(EIC_EY1_tmle1); se_EY1_tmle1
se_EY0_tmle1 <- (1/sqrt(N)) * sd(EIC_EY0_tmle1); se_EY0_tmle1
se_ATE_tmle1 <- (1/sqrt(N)) * sd(EIC_EY1_tmle1 - EIC_EY0_tmle1); se_ATE_tmle1
```

Question 5 - TMLE (a + b)

R Code: Step 5. Get 95% CI

```
# Compute the confidence intervals for  $E[Y^1]$  and  $E[Y^0]$ 
EY1_tmle1_CI_low <- EY1_tmle1 - 1.96 * se_EY1_tmle1; print(EY1_tmle1_CI_low)
EY1_tmle1_CI_high <- EY1_tmle1 + 1.96 * se_EY1_tmle1; print(EY1_tmle1_CI_high)

EY0_tmle1_CI_low <- EY0_tmle1 - 1.96 * se_EY0_tmle1; print(EY0_tmle1_CI_low)
EY0_tmle1_CI_high <- EY0_tmle1 + 1.96 * se_EY0_tmle1; print(EY0_tmle1_CI_high)

ATE_tmle1_CI_low <- ATE_tmle1 - 1.96 * se_ATE_tmle1; print(ATE_tmle1_CI_low)
ATE_tmle1_CI_high <- ATE_tmle1 + 1.96 * se_ATE_tmle1; print(ATE_tmle1_CI_high)
```

Question 5 - TMLE (c)

With this new fluctuation model, the only thing that changes is the parametric model that we build around the initial predictions to update them. In this case, we use a logistic regression including the intercept and use the clever covariates as weight instead of actual covariates.

R Code: Step 3. Fluctuation model 2

```
# Estimate parameters delta in the fluctuation models
delta1_tmle2 <- coef(glm(Y ~ offset(qlogis(Q1)),
                        family = 'binomial', weights = H1))
delta0_tmle2 <- coef(glm(Y ~ offset(qlogis(Q0)),
                        family = 'binomial', weights = H0))

# Targeting step
Q1_tmle2 <- plogis(qlogis(Q1) + delta1_tmle2/g)
Q0_tmle2 <- plogis(qlogis(Q0) + delta0_tmle2/(1-g))
```

Question 5 - TMLE (d)

R Code

```
# TMLE using tmle package (d)
library(tmle)

tmle_package <- tmle(Y = Y,
                     A = A,
                     W = X,
                     Q.SL.library = sl_lib,
                     g.SL.library = sl_lib,
                     family = 'binomial',
                     V.Q = 5,
                     V.g = 5)

summary(tmle_package)
```

You can specify different base learners and the number of folds for cross-validation in `SuperLearner` for `Q` and `g`.